

Linux Kernel Report

6.11 to 6.16

LG전자 이경식

kyungsiklee@yonsei.ac.kr

Contents

- `getrandom()` in the vDSO 6.11
- Dedicated bucket allocator 6.11
- BPF based task scheduling with `sched_ext` 6.12
- Real Time support 6.12
- Atomic block writes 6.13
- FUSE and `io_uring` 6.14
- The return of `RWF_UNCACHED` 6.14
- Mount notifications 6.15
- Device Memory TCP TX 6.16
- Automatic tuning for weighted interleaving 6.16

getrandom() in the vDSO 6.11

getrandom()

- 고빈도 난수 요청: 웹서버 session id, 게임, 블록체인 등 다양한 분야
- getrandom() 시스템 콜 -> mode switching 성능 저하
- 일부 앱은 성능 때문에 사용자 공간 RNG 사용
 - 보안 이슈: VM 포크, core dump 등으로 RNG 상태가 노출 가능 (재시드 필요)

vDSO 기반 getrandom()

- 성능 개선 해결책: virtual Dynamic Shared Object (vDSO) 기반의 getrandom()
 - e.g. gettimeofday()
- 커널의 Cryptographic Random Number Generator (CRNG): 암호학적으로 안전
- 기존 getrandom()은 유지, vDSO 버전은 glibc (patched)로 지원
- 성능 결과: 최대 15배 성능 향상 (예: uint32_t 생성 기준)

getrandom() in the vDSO 6.11

스레드별 상태 메모리 할당

- 해당 메모리 보호 플래그
 - VM_WIPEONFORK: fork() 시 상태 초기화
 - VM_DONTDUMP: core dump에서 제외
 - VM_DROPPABLE: 메모리 회수 시 zeroing (No swap)

vDSO와 커널 간 공유 구조체

```
struct vdso_rng_data {  
    u64 generation; // (변경 시 재시딩 필요)  
    u8 is_ready;    // 준비 상태  
};
```

vgetrandom()

- is_ready 필드를 확인 (준비되지 않으면 기존 getrandom() fallback)
- 공유 커널 구조체의 generation 값과 상태 영역 값을 비교해 필요 시 재시딩 (reseed)
- 난수생성: ChaCha20 스트림 암호 난수 생성
- 실행 중 상태 변경 감지 시 (e.g. fork) generation 재확인 후 다시 난수 생성

Dedicated bucket allocator 6.11

Heap Spraying이란?

- 공격자가 대량의 메모리 블록을 할당하고, 특정 데이터를 주입
- 커널이 해제된 메모리 위치에 이 데이터 중 하나를 재할당하길 기대

커널의 메모리 할당 방식

- kmalloc() 함수로 메모리 요청 -> 버킷(bucket) 단위로 할당
- 동일 크기의 객체들이 같은 버킷에 저장되어 공격자가 예측 가능
- kmalloc() 랜덤화
 - Linux 6.6부터 각 버킷을 16개로 분할
 - 요청 위치(call site) 기반으로 랜덤 버킷 선택
 - 공격자가 원하는 버킷을 맞추기 어려워짐 -> 확률적 방어

Dedicated bucket allocator 6.11

Dedicated Bucket Allocator

- 사용자 공간에서 제어 가능한 메모리 할당을 별도 풀로 분리
- `kmem_buckets *kmem_buckets_create(...)`: 전용버킷세트 생성
- `Void *kmem_buckets_alloc(...)`: 생성된 버킷에서 메모리 할당
- `msgsnd()` 시스템 호출: 메시지 구조체를 사용자 공간이 제어 -> 전용 버킷으로 분리
- 성능 영향: 메모리 파편화 증가 가능성 있음
- 보안성: 여전히 feng shui 공격 가능성 존재

```
static struct msg_msg *alloc_msg(size_t len)
{
@@ -50,7 +61,7 @@ static struct msg_msg *alloc_msg(size_t len)
size_t alen;
alen = min(len, DATALEN_MSG);
- msg = kmalloc(sizeof(*msg) + alen, GFP_KERNEL_ACCOUNT);
+ msg = kmem_buckets_alloc(msg_buckets, sizeof(*msg) + alen, GFP_KERNEL);
```

BPF based task scheduling with sched_ext 6.12

Linux 커널의 스케줄링 클래스

- 우선순위에 따라 계층화
- sched_ext는 기존 클래스(RT, DL, CFS)를 유지하면서, BPF를 통해 새로운 정책을 추가

Class	설명	특징
RT	실시간 작업을 위한 우선순위 기반 스케줄러 SCHED_FIFO나 SCHED_RR 정책 사용	고정 우선순위(1-99), 선점 가능 실시간 보장.
DL	데드라인 기반 스케줄링 SCHED_DEADLINE 정책	작업의 데드라인을 고려해 CPU 할당
CFS	일반 작업을 위한 공정 스케줄러 SCHED_NORMAL 정책	vruntime(virtual runtime)을 사용해 공정 분배
SCHED_EXT	BPF 기반 확장 스케줄러	동적 로드, 사용자 정의 알고리즘 CFS 위에 오버레이

BPF based task scheduling with sched_ext 6.12

SCHED_EXT

- 확장 가능한 스케줄링 프레임워크
- 커널 수정 없이 새로운 스케줄링 정책 실험 가능
- 특정 워크로드에 최적화된 정책 적용 가능
- BPF 기반으로 안전하고 효율적인 실행 가능

실제 적용 사례

- Meta(Facebook)의 사례
 - 데이터 센터 워크로드 최적화. sched_ext를 사용해 불필요한 인터럽트 최소화
 - 결과: 특정 워크로드에서 성능 10-20% 향상
- Google의 사례
 - 클라우드 인스턴스에서 워크로드별 스케줄링
 - sched_ext로 실시간 조정, 에너지 효율성 향상

Real Time support 6.12

PREEMPT_RT

- Linux를 실시간 운영 체제 (RTOS)로 만드는 핵심 기술
- 2005년부터 개발 시작 out-of-tree 패치로 20년 유지
- 2024년 9월 OSS에서 최종 merge 승인
- 주요 개발자: Sebastian Andrzej Siewior 외 다수

성능 비교

- worst-case latency : μs 수준 (e.g.: Ubuntu Pro 22.04에서 최대 $100\mu\text{s}$ 미만)
- Throughput: context switch 증가로 5-10% 감소 (응답 시간 10배 이상 개선)
- NASA 벤치마크: Hard real-time에서 안정적, 특정 하드웨어에서 latency spike 가능
- 오디오 처리: PREEMPT_RT + ALSA는 latency 1ms 미만, 일반 커널은 10ms 이상



Pic credit: <https://lwn.net/Articles/990985/>, Jon Corbet

Real Time support 6.12

주요 메커니즘

- Preemption 강화: Spinlocks를 rt-mutex로 대체, priority inheritance로 priority inversion 방지
- Threaded Interrupt Handlers: Hard IRQ를 최소화하고, softirq를 thread로 변환
- High-Resolution Timers: μ s 단위의 타이머 정확도
- No IRQ Nesting: IRQ 중첩 방지로 deterministic latency 보장

Challenges

- Overhead 증가: Threaded IRQ와 context switch로 CPU 사용량 증가
- Latency Spike: Raspberry Pi 등 특정 하드웨어에서 모듈 로딩이나 예상치 못한 지연 발생
- 코드 복잡성: printk 등 커널 서브시스템과의 호환성 문제로 mainline merge 지연
- 테스트 및 인증: Hard real-time 보장을 위한 광범위한 테스트 필요

Atomic block writes 6.13

Atomic block write

- DBMS는 torn write를 피하기 위해 double write 기법을 사용 -> 데이터 무결성 확보/write 성능 저하
- VFS: statx() 시스템 콜 확장, STATX_WRITE_ATOMIC 플래그로 atomic write 지원 여부 확인
- Block Layer: I/O 분할 방지 및 디바이스 제약 조건 상위 레이어로 전파
- Device Drivers: NVMe (암시적 지원, 제약 내), SCSI (특수 명령 필요)
- 하드웨어 의존
 - NVMe 1.4+: 16KB로 제한 많음 (/sys/block/<device>/queue/atomic_write_unit_max)
 - SCSI: 별도 명령을 통해 atomic write 지원
- 현재 한계: 단일 블록 쓰기만 지원, Direct I/O 전용
- 성능: 약 60-100% 개선

Atomic block writes 6.13

파일 시스템 지원

- EXT4: Direct I/O에서 단일 블록 (< 16KB) 지원, 멀티 블록 지원은 미구현
- XFS: 최근 대형 블록 크기 지원 추가

```
#define BUFFER_SIZE (16 * 1024) // 16KB, EXT4 블록 크기에 맞춤
#define ALIGNMENT (4096)        // 페이지 크기(4KB) 정렬

statx(AT_FDCWD, filename, 0, STATX_ALL, &statx_buf)
statx_buf.stx_attributes & STATX_ATTR_WRITE_ATOMIC

fd = open(filename, O_WRONLY | O_CREAT | O_DIRECT, 0644);

posix_memalign(&buffer, ALIGNMENT, BUFFER_SIZE)

written = pwritev2(fd, &(struct iovec){.iov_base = buffer, .iov_len = BUFFER_SIZE}, 1, 0, RWF_ATOMIC | RWF_DSYNC);
```

Atomic write code snippet (pwritev2() w/ RWF_ATOMIC flag)

FUSE and io_uring 6.14

FUSE와 io_uring의 통합

- Bernd Schubert (DDN Storage): 고성능 네트워크 스토리지 개발 중 FUSE의 한계에 직면
- io_uring을 활용한 통신 방식을 도입
- 기존 /dev/fuse 기반의 읽기/쓰기 인터페이스 대비 성능을 개선
- 시스템 호출 오버헤드 감소
 - io_uring을 사용하면 여러 요청을 한 번의 시스템 호출로 처리 -> 오버헤드 감소
- 요청 처리와 응답 결합
 - io_uring을 통해 이전 요청의 커밋과 새로운 요청의 페치를 동시에 처리
- CPU/NUMA 친화성:
 - 각 CPU 코어에 독립적인 io_uring 큐가 생성되어 요청이 해당 코어에 최적화된 큐로 처리

FUSE and io_uring 6.14

FUSE over io_uring 동작 방식

- 초기 설정
 - 사용자 공간의 FUSE 서버는 /dev/fuse 파일 디스크립터에 IORING_OP_URING_CMD 명령어 (SQE)를 제출하여 io_uring을 활성화
 - 초기 SQE는 FUSE_URING_REQ_REGISTER 하위 명령어로 전송되어 커널에 등록
- 요청 처리
 - 커널은 각 CPU 코어에 독립적인 io_uring 큐를 유지하며, 요청을 해당 큐에 추가
 - 사용자 공간은 CQE를 통해 요청을 수신하고, FUSE_URING_REQ_COMMIT_AND_FETCH를 사용하여 처리 결과를 커널에 반환하며 동시에 새 요청을 가져옴

파일 시스템 방식	초당 파일 생성 수 (creates/sec)	비고
기존 FUSE	2,000 ~ 2,500	/dev/fuse 기반, 병목 존재
io_uring 기반 FUSE	9,000 ~ 9,500	IORING_OP_URING_CMD 활용
XFS (native)	16,000 ~ 21,000	커널 내장 파일 시스템

성능 벤치마크: bonnie++ 테스트 기준:

The return of RWF_UNCACHED 6.14

RWF_UNCACHED FLAG

- Jens Axboe가 2019년에 처음 제안
- Buffered I/O와 direct I/O의 장점을 결합

문제 상황

- RAM보다 큰 파일에 대한 무작위 액세스와 같은 고부하 I/O 시나리오
- 커널의 kswapd 스레드가 캐시 정리에 과도한 CPU 사용
 - 사례: 32코어 시스템에서 28코어가 kswapd에 소모됨

동작

- Buffered I/O처럼 동작하지만, 작업 완료 후 페이지 캐시에서 즉시 제거
- 메모리 reclaim 없이 캐시 정리 가능

벤치마크 결과

- 고부하 읽기 작업에서 65% 더 빠른 속도와 CPU 사용량의 50% 감소

The return of RWF_UNCACHED 6.14

POSIX_FADV_NOREUSE와의 차이점

- posix_fadvise()의 FLAG
 - 파일의 특정 범위에 대해 cache 사용 힌트 -> 해당 데이터의 우선순위를 낮춤
 - Cache 제거: 커널의 캐시 관리 정책에 따름 (non-deterministic)
- RWF_UNCACHED
 - 작업 단위(per-IO), 비동기 I/O
 - io_uring/preadv2/pwritev2 사용

O_DIRECT와의 차이점

- O_DIRECT는 페이지 캐시를 완전히 우회하는 동기적 direct I/O

Mount notifications 6.15

fanotify()와의 통합

- fanotify(파일 및 디렉토리 변경 알림)를 확장
- 마운트 네임스페이스 파일 디스크립터를 fanotify 그룹에 등록
- 마운트 토폴로지 변경에 특화
- 지원되는 이벤트: mount, umount, moving mounts

기존 문제 해결

- 마운트 변경 감지: /proc/mounts나 /proc/<pid>/mountinfo 폴링

```
ns_fd = open("/proc/self/ns/mnt", )
fanotify_fd = fanotify_init()

fanotify_mark(fanotify_fd, ,ns_fd, )

// Monitoring mount events
```

Mount notifications 6.15

구현 세부 사항

- 새로운 시스템 콜이 도입되어 알림을 등록하고 처리
 - watch_mount()
 - mount_notify()

listmount()/statmount()

- 마운트 포인트의 계층 구조를 탐색하거나 특정 마운트 포인트 식별
- e.g. systemd나 docker가 mount 상태를 효율적으로 추적

```
num_mnts = listmount(0, mnt_ids, , )

for (;i < num_mnts;)
    statmount(mnt_ids[i], st, , )

printf("Mount ID: %llu\n", (unsigned long long)st->mnt_id)
```

Device Memory TCP TX 6.16

Device Memory TCP (devmem TCP)

- Direct Memory Access Buffer (DMA-BUF)를 활용
- 호스트 메모리를 거치지 않고 zero-copy 전송: NIC <-> Device Memory (e.g. GPU, AI 가속기)
- 대량 데이터 전송 시 호스트 메모리나 PCIe 대역폭을 효율적으로 사용
- 초기 구현 (RX path)가 6.12에 merge
 - Header split: NIC가 패킷 헤더와 페이로드를 분리
 - Payload만 DMA-BUF로 전송
 - Header는 여전히 호스트 메모리에서 처리 -> TCP 스택 오버헤드는 동일

Device Memory TCP TX

- MSG_ZEROCOPY: sendmsg() 호출 시 페이로드를 DMA-BUF에서 직접 참조
- Netlink 바인딩: netdev_bind_tx()로 DMA-BUF를 송신 큐에 효율적으로 매핑
- 성능: “~96% line rate at TCP level: 192 gbps bi-directional per NIC/GPU pair” (<https://netdevconf.org>)

Device Memory TCP TX 6.16

제한 및 요구사항

- TCP 소켓에만 구현
- Google GVE 드라이버 외의 NIC는 드라이버 수정이 필요
- 헤더 스플릿(RX) 및 zero-copy TX 지원 여부에 따라 성능 차이 발생
- PCIe 대역폭이 낮거나 GPU 메모리 접근 속도가 느리면 병목 발생 가능

테스트 환경

- 대부분의 벤치마크는 Google GVE 드라이버와 100Gbps NIC에서 수행됨
- 소비자급 NIC (e.g. Realtek)이나 저속 NIC에서는 성능 미지원 또는 저하
- 대량 데이터 전송 (64KB 이상)에서 최적화되며, 소규모 패킷에서는 이점이 제한적

Automatic tuning for weighted interleaving 6.16

Weighted Interleave

- NUMA 시스템에서 메모리 페이지 할당을 노드 간에 가중치에 따라 분배하는 정책
 - e.g. Node 0과 1에 가중치를 5:2로 설정
 - 기존 균등 분배 (round-robin) 방식보다 유연하게 대역폭과 지연 시간을 최적화
- 각 노드의 대역폭에 비례하는 가중치를 수동으로 설정
 - 복잡성: 대역폭 정보가 업데이트될 때 수동으로 조정
 - 비효율성: 부정확한 가중치 설정 -> 비효율적인 메모리 할당
 - 큰 가중치 비율 문제: e.g. 8912:1234:3의 경우 특정 노드가 거의 사용되지 않을 수 있음

Weighted Interleave Auto-Tuning

- 자동 가중치 계산: 부팅 시 또는 새로운 대역폭 정보 (e.g. 메모리 hot-plug in/out)가 제공될 때
- 모든 노드의 가중치를 자동으로 재계산

Automatic tuning for weighted interleaving 6.16

주요 특징

- 사용자 제어 가능: sysfs를 통해 weightiness 값 (32)을 조정하여 인터리빙 정책을 세밀하게 조정
- Weight 값 설정 기준: 작은 숫자로 설정해야 빠른 분산 가능
 - High weightness: 대역폭 비례성이 높아짐
 - Low weightness: 대역폭 비례성이 낮아짐

적용 시나리오

- Weighted Interleave Auto-Tuning은 특히 다중 메모리 노드를 사용하는 환경에 유용
 - 고성능 컴퓨팅(HPC) 서버, 클라우드 서버, 또는 CXL 메모리 사용 환경
- 메모리 대역폭을 최대한 활용하면서 관리자의 수동 개입을 최소화

Q&A

감사합니다!!

